

Reviewer 1

Novelty: 4

Rationale: The proposed method is similar to <https://arxiv.org/abs/2210.03493>; <https://aclanthology.org/2023.findings-acl.216/>

Feasibility: 6 (Feasible: Can be executed within the given constraints with some reasonable planning.)

Rationale: The experiments can be done with sufficient API access. The dataset collection needs some planning but is in general feasible to do. Setting up the vector database may take extra time.

Expected Effectiveness: 5 (Somewhat ineffective: There might be some chance that the proposed idea can work better than existing baselines but the improvement will be marginal or inconsistent.)

Rationale: The proposal is vague as it doesn't mention what's the final evaluation metric, and does not provide sufficient description of the compared baseline. The prompt in the direct prompt baseline is confusing to me as well. Overall it's hard to discuss the effectiveness.

Excitement: 4

Rationale: Given that the proposed method is vague, I am unsure about its contributions and effectiveness, and therefore I feel less excited about it.

Overall Score: 4 (Ok but not good enough, rejection for major AI conferences)

Rationale: The descriptions are confusing and I'm not really sure what's the focus or contribution. The title problem statement mentioned ensuring "diversity"/"high coverage" as the goal but doesn't describe how this is ensured in later sections. The "Test Case Examples" doesn't explain how the components in the "Step-by-Step Experiment Plan" are used.

Confidence: 3 (You are fairly confident that the evaluation is correct)

Reviewer 2

Novelty: 7

Rationale: Mapping natural language to custom applications is a hugely impactful capability, and doing so automatically is really interesting. I like the focus on autoprompting for these types of translations, as the task is feasible since it builds off some of the "few-shot prompting" that developers might normally do to add NL functionality, with a more automatic process that has real system checks/verifications (e.g., running the applications through containers). A related work from HCI tries to enable individual developers to add such NL functionality to their own applications via a DSL + NL program signatures (<https://jackieyang.me/reactgenie/>). This work is distinguished, as it would empower adding such NL functionality to any application, without changing the code.

Feasibility: 4

Rationale: The project infrastructure seems more difficult than simply choosing some prompting methods. It would be an iterative process choosing real example applications from Github, and developing the few-shot prompts manually to get a feel for this task. Then, some of the modules seem like 1-2 week tasks (Execution Module, Exploration, Storage) which I estimate would make the project more like 3 - 4 months to complete all modules AND to do the evaluations.

Expected Effectiveness: 7

Rationale: The baseline here is a zero-shot prompt, asking to do the NL intent and feeding in all the documentation of the API. Assuming the author is correct to say that such NL function mapping requires good few & diverse few-shot examples, I expect the method to work well. It uses a number of external systems to enrich the code dataset to give the LLM context and uses system errors to inform. So in some ways, Autoprompting is allowing an agent to make use of all these SWE tools for understanding the software, which then will allow it to maximize its understanding and better retrieve good few-shot examples for the task at hand.

Excitement: 7

Rationale: Seems like an impactful and ambitious outcome if completed. I am curious how such an approach fits into the conversation about general agents, which can leverage API/tool/functions calls. It's a little unclear from the toy example why existing function-calling models can't translate NL intents into.

Overall Score: 6 (Marginally above the acceptance threshold of major AI conferences)

Rationale: The results would be really exciting and the technical infrastructure to enable the Autoprompting agent would be impressive. However, I'm missing a bit of which cases will be really difficult for other generalist web/system agents, but where finding the few-shot examples for this task is really needed. Thus, the core idea of the method doesn't seem clarified enough to result in a really clear takeaway on the method.

Confidence: 3 (You are fairly confident that the evaluation is correct)

W Example Idea: Temporal Dependency Unfolding: Improving Code Generation for Complex Stateful Systems

Temporal Dependency Unfolding: Improving Code Generation for Complex Stateful Systems (Part 1)

1. Problem Statement: Generating code for complex, stateful systems or applications with intricate temporal dependencies remains challenging for current code generation models. Most existing approaches focus on generating individual functions or small code snippets without fully considering the temporal aspects and state changes in larger systems. This limitation hinders the applicability of AI-assisted programming in areas such as distributed systems, game development, and real-time applications.

2. Motivation: Many real-world applications require careful management of state over time. Existing code generation models struggle with capturing the full complexity of temporal dependencies and state changes in larger systems. A method that can effectively reason about and generate code for systems with complex temporal dependencies could significantly improve the applicability of AI-assisted programming in critical areas. Our proposed Temporal Dependency Unfolding method is inspired by how human developers approach complex system design, first identifying key states and their relationships before implementing the detailed logic.

3. Proposed Method: We propose Temporal Dependency Unfolding, a novel prompting technique that guides the model to generate code by explicitly reasoning about state changes and temporal relationships. The method consists of five steps:

1. **State Identification:** Prompt the model to identify key states and variables that change over time in the target system.
2. **Temporal Graph Construction:** Guide the model to create a conceptual graph of how these states evolve and interact over time.
3. **Staged Code Generation:** Generate code in stages, focusing on different temporal slices or state transitions in each stage.
4. **Consistency Verification:** After each stage, prompt the model to verify temporal consistency and make necessary adjustments.
5. **Integration:** Finally, guide the model to integrate the stage-wise generated code into a cohesive system, ensuring proper handling of all temporal dependencies.

4. Step-by-Step Experiment Plan:

1. Dataset Preparation:

- Create a dataset of programming tasks that involve complex temporal dependencies.
- Include tasks from three domains: 1) Multi-threaded applications, 2) Game logic, and 3) Distributed systems.
- For each domain, prepare 50 task descriptions, each with a clear specification of the desired functionality and temporal requirements.

2. Baseline Implementation:

- Implement two baseline methods:
 - Direct prompting: Simply provide the task description to the model and ask it to generate the code.
 - Chain-of-Thought (CoT) prompting: Append 'Let's approach this step-by-step:' to the task description.
- Use GPT-4 for both baselines.

Temporal Dependency Unfolding: Improving Code Generation for Complex Stateful Systems (Part 2)

4. Step-by-Step Experiment Plan (Continued):

3. Temporal Dependency Unfolding Implementation:

- Implement our proposed method with the following sub-steps for each task:
 - (a) State Identification: Prompt GPT-4 with 'Identify the key states and variables that change over time in this system:'.
 - (b) Temporal Graph Construction: Prompt with 'Create a conceptual graph showing how the identified states evolve and interact over time:'.
 - (c) Staged Code Generation: For each major state or transition identified, prompt with 'Generate code for the following state/transition: [state/transition]'.
 - (d) Consistency Verification: After each stage, prompt with 'Verify the temporal consistency of the generated code and suggest any necessary adjustments:'.
 - (e) Integration: Finally, prompt with 'Integrate the generated code segments into a cohesive system, ensuring proper handling of all temporal dependencies:'.

4. Evaluation Metrics:

- Correctness: Percentage of generated code that passes predefined test cases.
- Temporal Consistency: Manual evaluation of how well the code handles temporal dependencies (scale 1-5).
- Code Quality: Automated metrics like cyclomatic complexity and maintainability index.
- Execution Efficiency: Runtime performance on benchmark inputs.

5. Human Evaluation:

- Recruit 5 experienced developers to review a subset of 30 generated solutions (10 from each domain).
- They will rate the code on a scale of 1-5 for readability, maintainability, and correct handling of temporal dependencies.

6. Experiment Execution:

- For each task in the dataset:
 - (a) Generate solutions using both baseline methods and our Temporal Dependency Unfolding method.
 - (b) Apply all evaluation metrics to the generated solutions.
 - (c) Collect human evaluations for the subset of solutions.

7. Analysis:

- (a) Compare the performance of Temporal Dependency Unfolding against the baselines across all metrics.
- (b) Analyze the effectiveness of each step in our method (State Identification, Temporal Graph Construction, etc.) by examining intermediate outputs.
- (c) Identify patterns in tasks where our method shows significant improvement or underperforms.
- (d) Correlate automated metrics with human evaluations to validate their reliability.

Temporal Dependency Unfolding: Improving Code Generation for Complex Stateful Systems (Part 3)

5. Test Case Examples:

- **Test Case 1:**

- **Baseline Prompt Input (Direct Prompting):** Generate Python code for a simple multi-threaded producer-consumer system with a shared buffer. The producer should generate random numbers and add them to the buffer, while the consumer should remove and process these numbers. Implement proper synchronization to avoid race conditions.
- **Baseline Prompt Expected Output (Direct Prompting):** [Python code for a simple producer-consumer system]
- **Proposed Prompt Input (Temporal Dependency Unfolding; Step 1: State Identification):** For a multi-threaded producer-consumer system with a shared buffer, identify the key states and variables that change over time in this system:
- **Proposed Prompt Expected Output (Temporal Dependency Unfolding; Step 1: State Identification):** [List of key states and variables]
- **Proposed Prompt Input (Temporal Dependency Unfolding; Step 2: Temporal Graph Construction):** Create a conceptual graph showing how the identified states evolve and interact over time for the producer-consumer system:
- **Proposed Prompt Output (Temporal Dependency Unfolding; Step 2: Temporal Graph Construction):** [Conceptual graph of state evolution and interactions]
- **Proposed Prompt Input (Temporal Dependency Unfolding; Step 3: Staged Code Generation):** Generate code for the producer functionality in the producer-consumer system, focusing on its interaction with the buffer and synchronization mechanisms:
- **Proposed Prompt Output (Temporal Dependency Unfolding; Step 3: Staged Code Generation):** [Python code for producer functionality]
- **Proposed Prompt Input (Temporal Dependency Unfolding; Step 4: Consistency Verification):** Verify the temporal consistency of the generated producer code and suggest any necessary adjustments:
- **Proposed Prompt Output (Temporal Dependency Unfolding; Step 4: Consistency Verification):** [Verification and adjustment suggestions]
- **Proposed Prompt Input (Temporal Dependency Unfolding; Step 5: Integration):** Integrate the generated producer code with a consumer and main control logic to create a complete producer-consumer system, ensuring proper handling of all temporal dependencies:
- **Proposed Prompt Output (Temporal Dependency Unfolding; Step 5: Integration):** [Complete Python code for producer-consumer system]
- **Explanation:** The Temporal Dependency Unfolding method produces a more comprehensive and robust solution compared to the baseline. It explicitly handles temporal dependencies, includes proper synchronization, and provides mechanisms for graceful termination. The staged approach allows for better handling of edge cases and improved overall system design.

6. Fallback Plan: If the Temporal Dependency Unfolding method does not show significant improvement over the baselines, we can pivot the project in several ways. First, we could conduct an in-depth analysis of where and why the method fails, which could provide valuable insights into the limitations of current language models in handling temporal reasoning tasks. This analysis could involve examining the intermediate outputs (state identification, temporal graphs) to understand where the reasoning breaks down. Second, we could explore combining our method with other techniques, such as retrieval-augmented generation, to see if providing relevant examples improves performance. Third, we could focus on developing a new evaluation framework specifically designed to assess temporal reasoning in code generation, which could be a valuable contribution to the field even if our primary method doesn't outperform baselines. Lastly, we could investigate whether the method performs better on certain types of temporal dependencies or specific programming domains, which could lead to a more targeted approach for improving code generation in those areas.